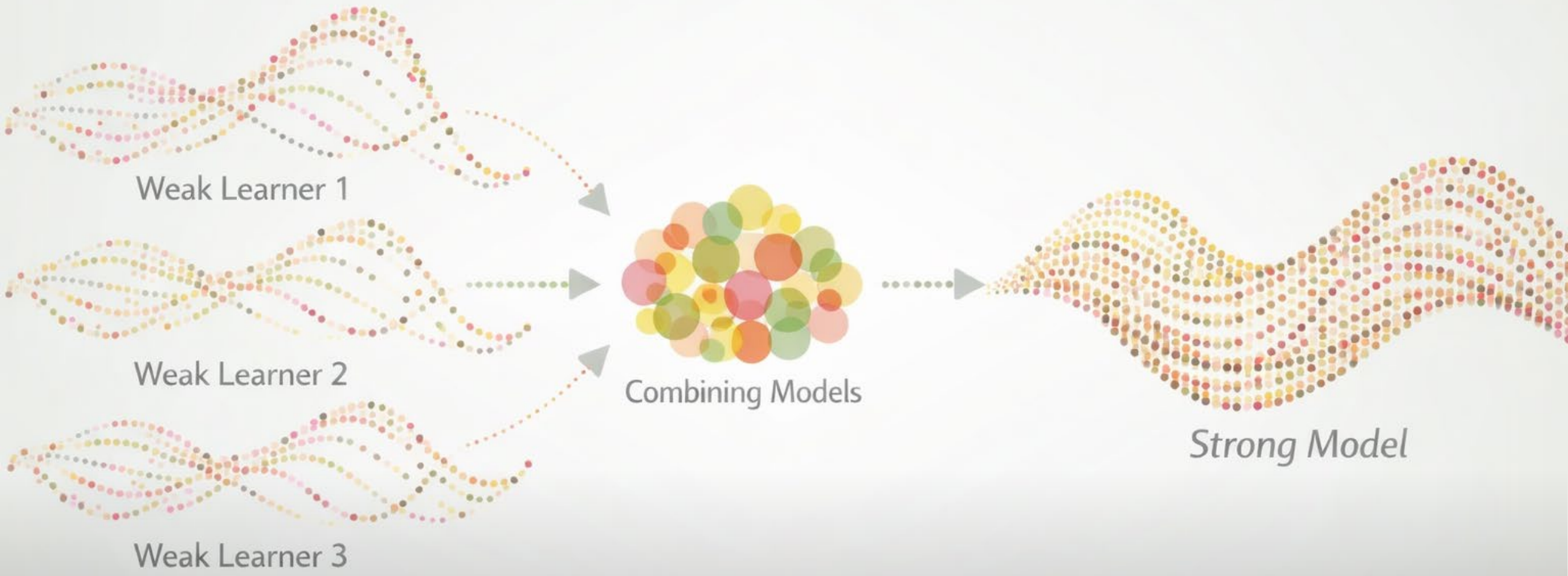

Weak Learner 1
Weak Learner 2
Weak Learner 3
Combining Models
Strong Model

# BOOSTING

# BOOSTING

- Boosting is a technique that was developed to improve **weak learners**
  - **Models** that are not much better than random guessing
  - Simple (few tuning parameters) and fast to train
  - Decision "stumps" are commonly used weak learners
- Idea: boosting methods train models sequentially and try to "***fix the mistakes***" of the preceding model

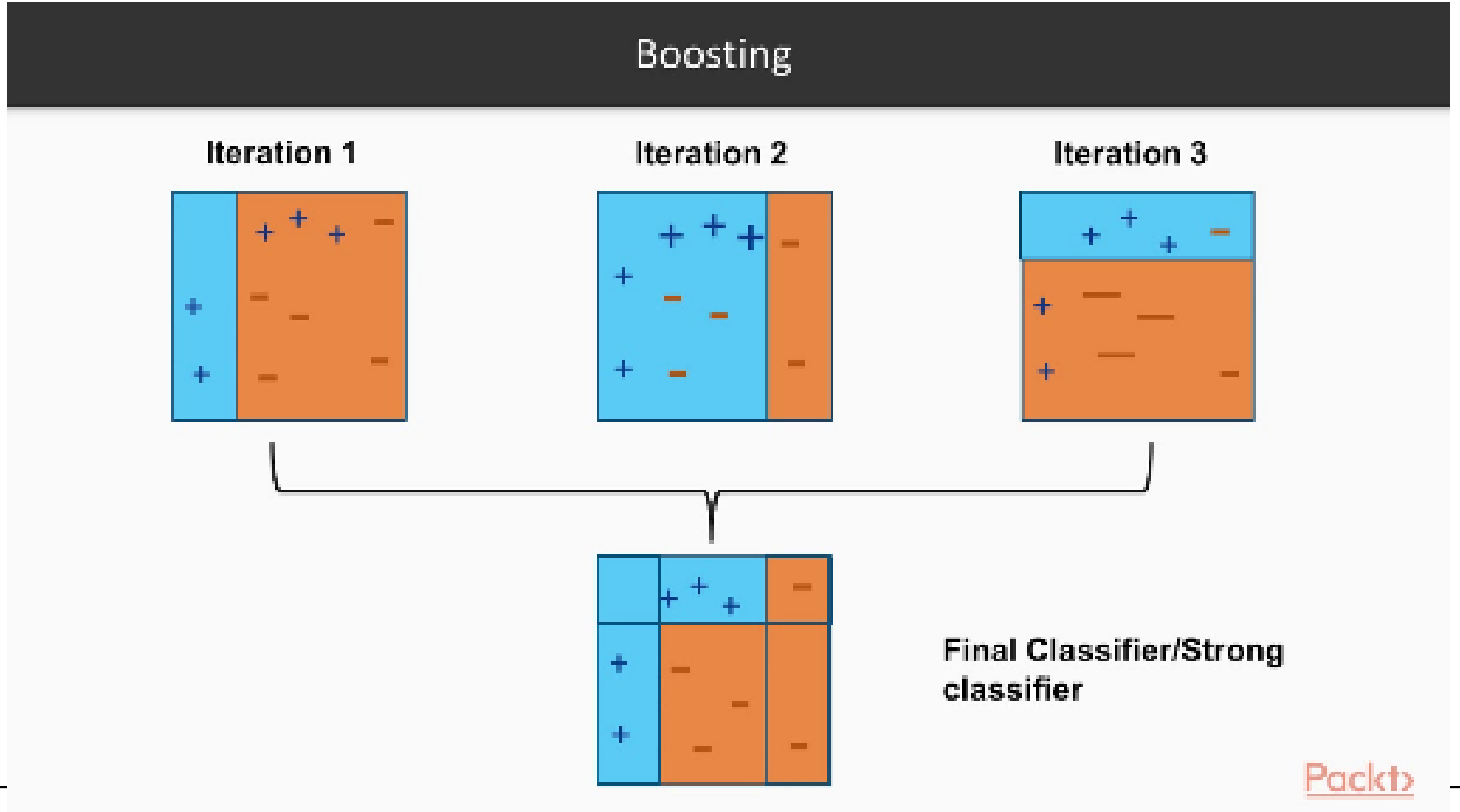
# FOUNDATIONAL BOOSTING ALGORITHMS

- Adaptive Boosting (AdaBoost) — increase the weight of misclassified observations
- Gradient Boosting — learn and then correct the mistakes by predicting residuals
  - Modern implementations:
    - XGBoost (Extreme gradient boosting)
    - LightGBM (Light gradient boosting machine)
    - Catboost (Categorical boosting)

# ADABOOST

- Adaptive Boosting (AdaBoost)
- Introduced in 1997 by Yoav Freund and Robert Schapire
- The main idea is to “pay more attention” to instances that were misclassified or had large errors
  - Focus learning on difficult instances
  - “Study” topics you missed last time
- Adaboost was written as a binary classification method, though multiclass and regression algorithms have been adapted

# ADABOOST


Boosting
Iteration 1
Iteration 2
Iteration 3
Final Classifier/Strong classifier
Packt>

# ADABOOST ALGORITHM

1. Assign each training point a weight: $w_i = 1/n$

   $i$ indexes observations

   *F(x)* is the ensemble's running prediction, starting at F(x) = 0 and updated in step 5

2. Train a weak learner (e.g., a stump) and predict

3. Compute the weighted error rate: $\epsilon_j = \frac{\sum w_i I(y_i \neq \hat{y}_i)}{\sum w_i}$

   $j$ indexes classifiers

4. Compute the classifier weight: $\alpha_j = \frac{1}{2}\log\left(\frac{1-\epsilon_j}{\epsilon_j}\right)$

   $\alpha_j$ is larger for more accurate models and sets their vote in the ensemble

# ADABOOST

5. Update the model prediction, scaled by a learning rate to regularize training

   $F_{\mathrm{j}}(x) = F_{\mathrm{j}-1}(x) + \lambda \cdot \alpha_j \cdot \hat{y}_j(x)$

   - A low learning rate ($\lambda < 1$) makes updates to the prediction smaller
   - This can reduce overfitting by making adjustments more gradual

# ADABOOST

6. Update the instance weights, then normalize

   $w_i^{(j+1)} = w_i^{(j)} \cdot \exp(\alpha_j)$ if $y_i \neq \hat{y}_i$

   $w_i^{(j+1)} = w_i^{(j)} \cdot \exp(-\alpha_j)$ if $y_i = \hat{y}_i$

   Normalize by dividing by $\sum w_i$

7. Repeat steps 2–6: the next classifier minimizes the weighted error, which now penalizes errors on high-weight instances

   The model focuses on instances misclassified in the previous step

   The weights enter training directly: a mistake on a high-weight point costs more in the weighted error

   If a learner cannot use sample weights, resample the training data in proportion to the weights

# ADABOOST PREDICTIONS

1. Run the input x through every trained weak learner to get its prediction $\hat{y}_j(x) = \pm 1$
   - No weights are updated at prediction time; only the saved learners and their α weights are used
2. Combine them as a weighted vote: $F(x) = \Sigma_j \lambda \cdot \alpha_j \cdot \hat{y}_j(x)$
   - This is the F built up during training (one $\lambda \cdot \alpha_j \cdot \hat{y}_j(x)$ term per round); larger $\alpha_j$ gets more say
3. Predict the class from the sign of the vote: $\hat{y} = \text{sign}(F(x))$
   - $\lambda > 0$ does not change the predicted class; $|F(x)|$ indicates confidence (near 0 is a near-tie)

# ADABOOST SUMMARY

1. Initialize weights: $w_i^{(1)} = 1/n$, F(x) = 0
   $i$ indexes observations, $j$ indexes classifiers
2. Train a weak learner
3. Compute weighted error: $\epsilon_j = \frac{\sum w_i^{(j)} I(y_i \neq \hat{y}_i)}{\sum w_i^{(j)}}$
4. Compute classifier weight: $\alpha_j = \frac{1}{2}\log\left(\frac{1-\epsilon_j}{\epsilon_j}\right)$
5. Update the ensemble with a learning rate: $F_j(x) = F_{j-1}(x) + \lambda \cdot \alpha_j \cdot \hat{y}_j(x)$
6. Update and normalize weights:
   $w_i^{(j+1)} = w_i^{(j)} \cdot \exp(\alpha_j)$ if $y_i \neq \hat{y}_i$
   $w_i^{(j+1)} = w_i^{(j)} \cdot \exp(-\alpha_j)$ if $y_i = \hat{y}_i$
7. Repeat steps 2–6 for each round, then predict ŷ = sign(F(x))


AdaBoost Sequential Predictions & Instance Error Tracking (Noisy Two Moons)
Iteration 1 | Acc: 84.5% (31 errors)
Iteration 2 | Acc: 84.5% (31 errors)
Iteration 3 | Acc: 90.5% (19 errors)
Iteration 5 | Acc: 91.0% (18 errors)
Iteration 10 | Acc: 91.0% (18 errors)
Iteration 20 | Acc: 96.0% (8 errors)
Class 0 - Correct
Class 0 - Error
Class 1 - Correct
Class 1 - Error
Ensemble Boundary

# GRADIENT BOOSTING

- Gradient Boosting directly targets the residuals (regression) or gradient of the loss function (classification)
- At each step, a new weak learner is fit to the residual or pseudo-residuals

# GRADIENT BOOSTING

1.Train a weak learner and make predictions

2. Calculate the residuals (regression) as:

$$Residual = Actual\ Target - Current\ Prediction$$

3.Train a new weak learner that tries to predict the residuals from the previous step

4. Update the Predictions as:

$$\text{New Predictions} = \text{Current Prediction} + \lambda \times (\text{Predictions of New Model})$$

"Predicted Residuals"

Learning Rate

5. Repeat for a predefined number of iterations or until a stopping criterion is reached

6. Final prediction: $F_M(x) = F_0(x) + \sum_{m=1}^{M} \lambda h_m(x)$

# EXAMPLE (WITH $\lambda = 1$)

| **Current Target** | **Current Predictions** | | | | | |
|---|---|---|---|---|---|---|
| **Y** | Yhat-1 | Resid-1 | Yhat-2 | Resid-2 | Yhat-3 | FinalPrediction |
| **0.05** | 0.12 | −0.07 | −0.09 | 0.02 | 0.01 | 0.04 |
| **0.59** | 0.53 | 0.07 | 0.04 | 0.03 | −0.07 | 0.50 |
| **0.17** | 0.12 | 0.04 | 0.04 | 0.00 | 0.01 | 0.17 |
| **−0.07** | 0.12 | −0.19 | −0.09 | −0.10 | 0.01 | 0.04 |
| **0.34** | 0.12 | 0.22 | 0.16 | 0.06 | 0.01 | 0.29 |

# EXAMPLE IN A TABLE

**Current Target**

| Y | Yhat-1 | **Resid-1** | Yhat-2 | Resid-2 | Yhat-3 | FinalPrediction |
|---|---|---|---|---|---|---|
| 0.05 | 0.12 | **−0.07** | −0.09 | 0.02 | 0.01 | 0.04 |
| 0.59 | 0.53 | **0.07** | 0.04 | 0.03 | −0.07 | 0.50 |
| 0.17 | 0.12 | **0.04** | 0.04 | 0.00 | 0.01 | 0.17 |
| −0.07 | 0.12 | **−0.19** | −0.09 | −0.10 | 0.01 | 0.04 |
| 0.34 | 0.12 | **0.22** | 0.16 | 0.06 | 0.01 | 0.29 |

# EXAMPLE IN A TABLE

| | | Current Target | Current Predictions | | | |
|---|---|---|---|---|---|---|
| Y | Yhat-1 | Resid-1 | Yhat-2 | Resid-2 | Yhat-3 | FinalPrediction |
| 0.05 | 0.12 | −0.07 | −0.09 | 0.02 | 0.01 | 0.04 |
| 0.59 | 0.53 | 0.07 | 0.04 | 0.03 | −0.07 | 0.50 |
| 0.17 | 0.12 | 0.04 | 0.04 | 0.00 | 0.01 | 0.17 |
| −0.07 | 0.12 | −0.19 | −0.09 | −0.10 | 0.01 | 0.04 |
| 0.34 | 0.12 | 0.22 | 0.16 | 0.06 | 0.01 | 0.29 |

# EXAMPLE IN A TABLE

| Y | Yhat-1 | Resid-1 | Yhat-2 | **Resid-2** | Yhat-3 | FinalPrediction |
|---|---|---|---|---|---|---|
| 0.05 | 0.12 | −0.07 | −0.09 | **0.02** | 0.01 | 0.04 |
| 0.59 | 0.53 | 0.07 | 0.04 | **0.03** | −0.07 | 0.50 |
| 0.17 | 0.12 | 0.04 | 0.04 | **0.00** | 0.01 | 0.17 |
| −0.07 | 0.12 | −0.19 | −0.09 | **−0.10** | 0.01 | 0.04 |
| 0.34 | 0.12 | 0.22 | 0.16 | **0.06** | 0.01 | 0.29 |

**Current Target**

# EXAMPLE IN A TABLE

| | | | | **Current Target** | **Current Predictions** | |
|---|---|---|---|---|---|---|
| Y | Yhat-1 | Resid-1 | Yhat-2 | Resid-2 | Yhat-3 | FinalPrediction |
| 0.05 | 0.12 | −0.07 | −0.09 | 0.02 | 0.01 | 0.04 |
| 0.59 | 0.53 | 0.07 | 0.04 | 0.03 | −0.07 | 0.50 |
| 0.17 | 0.12 | 0.04 | 0.04 | 0.00 | 0.01 | 0.17 |
| −0.07 | 0.12 | −0.19 | −0.09 | −0.10 | 0.01 | 0.04 |
| 0.34 | 0.12 | 0.22 | 0.16 | 0.06 | 0.01 | 0.29 |

# EXAMPLE IN A TABLE

| Y | Yhat-1 | Resid-1 | Yhat-2 | Resid-2 | Yhat-3 | **Final Prediction** |
|---|---|---|---|---|---|---|
| 0.05 | 0.12 | −0.07 | −0.09 | 0.02 | 0.01 | **0.04** |
| 0.59 | 0.53 | 0.07 | 0.04 | 0.03 | −0.07 | **0.50** |
| 0.17 | 0.12 | 0.04 | 0.04 | 0.00 | 0.01 | **0.17** |
| −0.07 | 0.12 | −0.19 | −0.09 | −0.10 | 0.01 | **0.04** |
| 0.34 | 0.12 | 0.22 | 0.16 | 0.06 | 0.01 | **0.29** |

# TUNING PARAMETERS

- **Number of estimators**
  - Too few → underfitting; too many → overfitting
  - Early stopping helps choose the number
- **Learning rate ($\lambda$)**
  - Shrinkage parameter that scales each tree's contribution
  - Smaller → more trees, better generalization
  - Larger → fewer trees, risk of overfitting or instability
- **Trade-off**
  - Low rate + many trees → slower, more stable
  - High rate + few trees → faster, more aggressive

# GRADIENT BOOSTING FOR CLASSIFICATION

- The concept of "residuals" is a little different in the classification setting
  - The residual or pseudo residuals are the gradient of the loss function with respect to the predictions.
- For example, in binary classification:
  - Log loss or binary cross-entropy loss:

$$L(y, p) = -y\log(p) - (1 - y)\log(1 - p)$$

* $y$ is the actual label (0 or 1)

* $p$ is the predicted probability of the instance being in class 1

# GRADIENT BOOSTING FOR CLASSIFICATION

- The gradient of the loss function with respect to p:

$$\frac{\partial L}{\partial p} = \frac{p - y}{p(1 - p)}$$

- This is simplified by working with the log odds,

  $z = \log\left(\frac{p}{1-p}\right)$, where $p = \frac{1}{1+e^{-z}}$

- The gradient with respect to the log odds is:

$$\frac{\partial L}{\partial z} = p - y$$

# IMPLEMENTATIONS OF GRADIENT BOOSTING

- **Scikit-Learn**
  - GradientBoostingClassifier / GradientBoostingRegressor
  - Can be slower on large datasets
  - HistGradientBoosting provides a faster, histogram-based implementation
- **XGBoost (Extreme Gradient Boosting)**
  - Highly optimized and efficient
  - Supports L1 and L2 regularization
  - Handles missing values automatically
  - Parallelized and widely used in industry
- **LightGBM (Light Gradient Boosting Machine)**
  - Very fast for large datasets
  - Histogram-based learning for efficiency
  - Leaf-wise tree growth strategy
  - Native handling of categorical features
  - Memory efficient
- **CatBoost**
  - Strong performance with minimal tuning
  - Excellent handling of categorical variables
  - Reduces preprocessing requirements

# INSTALLING BOOSTING LIBRARIES

## Using pip

### XGBoost

```
pip install xgboost
```

### LightGBM

```
pip install lightgbm
```

### CatBoost

```
pip install catboost
```

## Using conda

### XGBoost

```
conda install -c conda-forge py-xgboost
```

### LightGBM

```
conda install -c conda-forge lightgbm
```

### CatBoost

```
conda install -c conda-forge catboost
```